

可靠数据传输

1.rdt协议

1.1 rdt1.0

在可靠信道上的可靠数据传输

- 下层的信道是完全可靠的：没有比特差错；没有分组丢失
- 发送方和接受方的FSM：发送方将数据发送到下层信道；接收方从下层信道接受数据

1.2 rdt2.0

具有比特差错的信道

- 下层信道可能非出错：将分组中的比特翻转：用校验和来检测差错
- 问题：如何从差错中恢复？

确认 (ACK)：接收方显示地告诉发送方分组已被正确接受

否认确认 (NAK)：接收方显示地告诉发送方分组发生了差错

发送方受到NAK后，发送方重传分组

- rdt2.0中的新机制：采用差错控制编码进行差错检测

1.3 rdt2.1

解决了rdt2.0中的致命缺陷：ACK和NAK可能出错！

这里引入了停止等待协议，对packet进行编号。

- 发送方获得来自上层的数据后，packet(编号为0)发送到接收方，接收方接收到分组后：
 - 1.如果没有损坏，回送ACK分组，并等待编号为1的分组
 - 2.如果损坏，回送NAK分组，并继续等待编号为0的分组
- 发送方接收到回送分组后：
 - 1.如果没有损坏，且为ACK分组，无任何动作，等待上层协议
 - 2.如果没有损坏，且为NAK分组，重发编号为0的分组
 - 2.如果损坏，无论是ACK还是NAK，重发编号为0的分组
- 接收方收到分组后（接收方不知道自己的ACK或NAK在传输过程中是否损坏）：
 - 1.如果没有损坏，且为编号1的分组，回送ACK分组，且等待编号为0的分组（循环）
 - 2.如果没有损坏，且为编号为0的分组，回送ACK分组，且等待编号为1的分组
 - 3.如果损坏，回送NAK分组，等待编号为0的分组

1.4 rdt2.2

功能同rdt2.1,但是不用NAK，对ACK进行编号

“答非所问”！

- 发送方获得来自上层的数据后，封包（编号为0）发送至接收方，接收方接收到分组后：
 - 1、如果没有损坏，回送ACK0分组，并等待编号为1的分组

- 2、如果损坏，回送ACK1分组，并继续等待编号为0的分组
- 发送方接收到回送分组后：
 - 1、如果没有损坏，且为ACK0分组，无任何动作，等待上层协议
 - 2、如果没有损坏，且为ACK1分组，重发编号为0的分组
 - 3、如果损坏，无论是ACK1还是ACK0，重发编号为0的分组
- 接收方收到分组后（接收方不知道自己的ACK分组在传输过程中是否损坏）：
 - 1、如果没有损坏，且为编号1的分组，回送ACK1分组，且等待编号为0分组（循环）
 - 2、如果没有损坏，且为编号0的分组，回送ACK0分组，且等待编号为1分组
 - 3、如果损坏，回送ACK1分组，等待编号为0分组

1.5 rdt3.0（比特交替协议）

具有比特差错和分组丢失的信道

新的假设：下层信道可能会丢失分组（数据或ACK）

- 会死锁
 - 机制还不够处理这种状况
- 检验和
- 序列号
- ACK
- 重传
- 方法：发送方等待ACK一段合理的时间，“**超时重传**”！
 - 问题：如果分组或ACK只是被延迟了：重传会导致数据重复，但这并不是问题
 - 需要一个倒计时定时器

过早超时（延迟的ACK）也能正常工作，但是效率较低，一般的分组和确认是重复的

设置一个合理的超时时间！

1.6 rdt3.0的性能

在链路容量比较大时，rdt3.0性能很差

2.流水线协议

2.1 简述

1. 作用：提高链路利用率。
 2. 流水线：允许发送方在未得到对方确认的情况下一一次发送多个分组。
- 必须增加序号的范围：单纯用0/1已经不能满足要求，需要用多个bit表示分组的序号
 - 在发送方/接收方要有缓冲区

发送方缓冲：未得到响应，可能需要重传；便于检错重发和超时重发

接收方缓冲：上层用户取用数据的速率不等于接收到的数据速率，接收到的数据可能乱序，对抗两个速率的不同

3. 两种通用的流水线协议：回退N步（GBN）和选择重传（SR）

2.2 通用：滑动窗口协议 (Sliding Window)

SW=1,RW=1: 停止等待协议

SW>1,RW=1: GBN

SW>1,RW>1: SR

2.2.1 发送缓冲区

1. 形式：内存中的一个区域，落入缓冲区的分组可以发送
2. 功能：用于存放已发送，但是没有得到确认的分组
3. 必要性：需要重发时可用
4. 大小：规定了一次最多可以发送多少个未经确认的分组
 - 对于停止等待协议=1
 - 对于流水线协议>1，有上限，链路利用率不能大于100%
5. 发送缓冲区中的分组：
 - 未发送的：落入发送缓冲区的分组，可以连续发送出去
 - 已经发送出去的、等待对方确认的分组：发送缓冲区的分组自由得到确认才能删除

2.2.2 发送窗口

1. 定义：发送缓冲区的一个子集

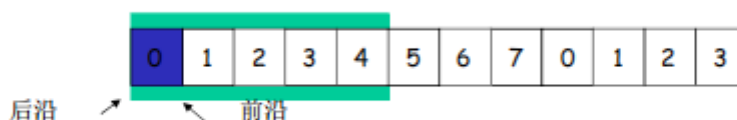
那些已发送但是未经确认分组的序号构成的空间

2. 大小：发送窗口的最大值 $\leq$ 发送缓冲区的值

一开始：没有发送任何一个分组

- 后沿=前沿
 - 之前为发送窗口的尺寸=0
3. 前沿移动：每发送一个分组，前沿前移一个单位

发送窗口前沿移动的极限：不能超过发送缓冲区

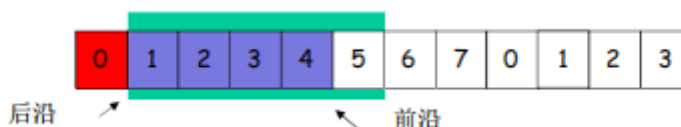


4. 后沿移动：

条件：收到老分组（后沿）的确认

结果：发送缓冲区罩住新的分组，来了分组可以发送

移动的极限：不能超过前沿



2.2.3 接收窗口

1. 接收窗口 (receiving window) = 接收缓冲区
2. 功能: 控制哪些分组可以接收
 - 只有收到的分组序号落入接收窗口内才可以接收
 - 若序号在接收窗口之外, 则丢弃
3. 大小:
 - 接收窗口 $W_r = 1$, 则只能顺序接收
 - 接收窗口 $W_r > 1$, 则可以乱序接收; 但是提交给上层的分组要按序
4. 接收窗口的滑动和发送确认
 - 滑动:
 - 低序号的分组到来, 接收窗口移动
 - 高序号的分组到来, 缓存但不交付 (因为要实现 rdt, 不能乱序), 不滑动
 - 发送确认:
 - 接收窗口尺寸=1: 发送连续收到的最大的分组确认 (累计确认)
 - 接收窗口尺寸>1: 收到分组, 发送那个分组的确认 (非累计确认)

2.2.4 两个窗口的互动

1. 正常情况下:
 - 发送窗口:
 - 有新的分组落入发送缓冲区范围, 发送->前沿滑动
 - 来了老的低序号分组的确认->后沿向前滑动->新的分组可以落入发送缓冲区的范围
 - 接收窗口:
 - 收到分组, 落入到接收窗口范围内, 接收
 - 是低序号, 发送确认给对方
 - 发送端上面来了分组->发送窗口滑动->接收窗口滑动->发确认
2. 异常情况下GBN的2窗口互动:
 - 发送窗口:
 - 新分组落入发送缓冲区范围, 发送->前沿滑动
 - 超时重发机制让发送端将发送窗口中的所有分组发送出去
 - 来了老分组的重复确认->后沿不向前滑动->新的分组无法落入缓冲区的范围 (此时如果发送缓冲区有新的分组可以发送)
 - 接收窗口:
 - 收到乱序分组, 没有落入到接收窗口范围内, 抛弃
 - (重复) 发送老分组的确认, 累计确认
3. 异常情况下SR的2窗口互动
 - 发送窗口:
 - 新分组落入发送缓冲区范围, 发送->前沿滑动

超时重发机制让发送端将超时的分组重新发送出去

来了乱序分组的确认->后沿不向前滑动->新的分组无法落入发送缓冲区的范围（此时如果发送缓冲区有新的分组可以发送）

- 接收窗口：

收到乱序分组，落入到接收窗口范围内，接收

发送该分组的确认，单独确认

2.3 GBN协议和SR协议的异同

1. 相同之处：

- 发送窗口 >1
- 一次能够发送多个未经确认的分组

2. 不同之处：

- GBN：接收窗口尺寸 $=1$

接收端：只能顺序接收

发送端：一旦一个分组没有发送成功，如：0, 1, 2, 3, 4；加入1未成功，2, 3, 4都发出去了，要返回1再重发

- SR：接收窗口尺寸 >1

接收端：可以乱序接收

发送端：发送0, 1, 2, 3, 4，一旦1未成功，2, 3, 4已经发送，无需重发，选择性发送1

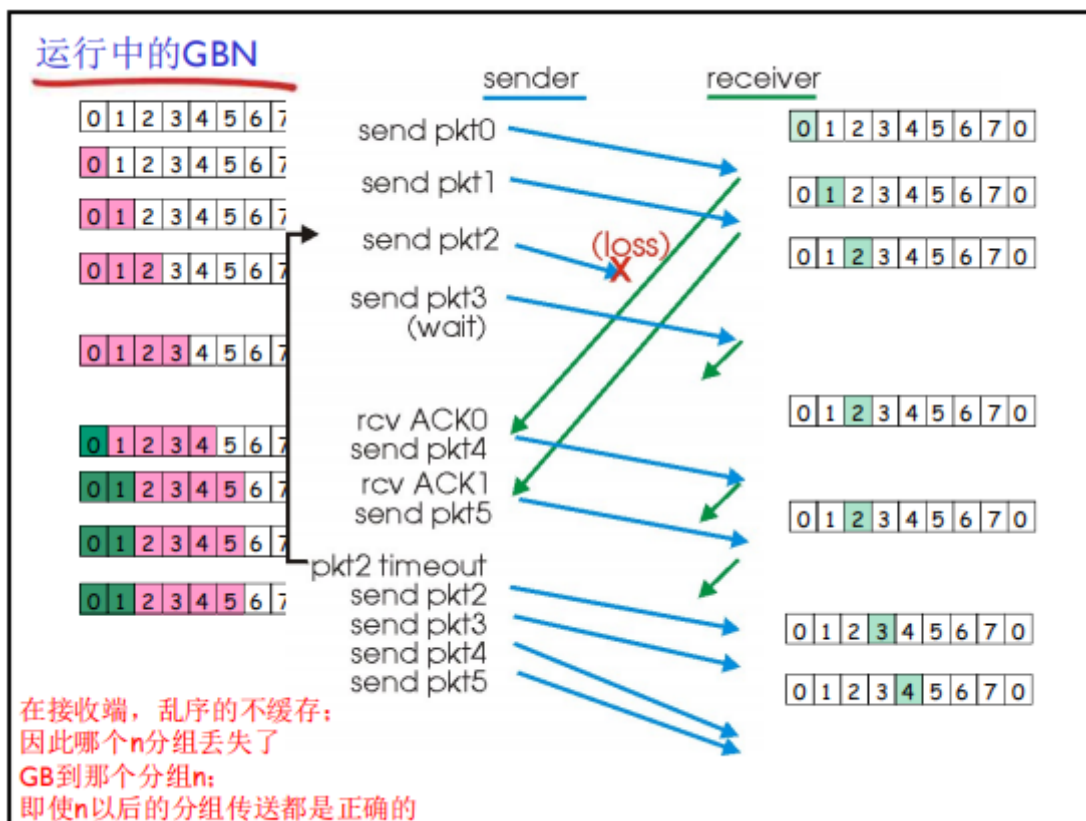
2.4 总结

1. Go-Back-N:

- 发送端最多在流水线中有N个未经确认的分组
- 接收端只是发送累计型确认（cumulative ack）；接收端如果发现gap，不确认新到来的分组
- 发送端拥有对最老的未经确认分组的定时器
只需设置一个定时器
当定时器到达时，重传所有未确认分组

2. Selective Repeat:

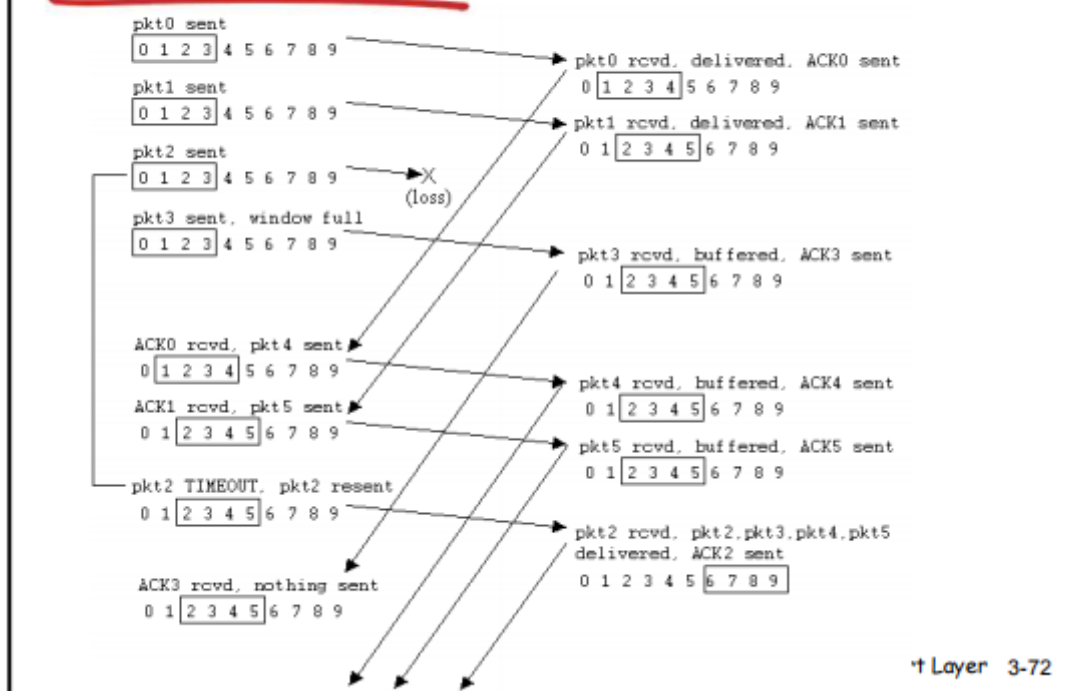
- 发送端最多在流水线中有N个未经确认的分组
- 接收方对每个到来的分组单独确认（individual ack），非累计确认
- 发送方为每个未经确认的分组保持一个定时器
当定时器到时，只是重复到时的未经确认的分组



3.选择重传(SR)

1. 接收方对每个正确接收的分组，分别发送ACKn（非累计确认）
 - 接收窗口>1：可以缓存乱序的分组
 - 最终将分组按顺序交付给上层
2. 发送方只对那些没有收到ACK的分组进行重发——选择性重发
 - 发送方为每一个未确认的分组设定一个定时器:timeout(n)
3. 发送窗口的最大值（发送缓冲区）限制发送未确认分组的个数

选择重传SR的运行



4.GBN和SR的对比

	GBN	SR
优点	简单，所需资源少（接收方一个缓存单元）	出错时，重传一个代价小
缺点	一旦出错，回退N步代价大	复杂，所需要资源多（接收方多个缓存单元）

1. 适用范围：

- 出错率低：适合GBN
- 链路容量大（延迟大、带宽大）：适合SR

2. 窗口的最大尺寸：

- GBN: $2^n - 1$
- SR: 2^{n-1}

n 是比特位数